Lista de exercícios 3

1. Crie uma função para buscar um valor **v** em uma pilha **P** fazendo uso de uma pilha auxiliar **AUX**. Observações:
   - Protótipo da função:
     `bool buscaValor(Pilha *p, int valor);`
   - Não é permitido o uso de vetores auxiliares.
   - Após a chamada da função, a pilha **P** deve conter os mesmos elementos, os valores devem ser preservados.
   - Implementação da função:
     i. Desempilhar os elementos de **P** e empilhar em **AUX.** Dentro de um laço de repetição:
        - Desempilhar um elemento da pilha **P** e empilhar na pilha **AUX**.
        - Verificar se o elemento desempilhado é igual **v.** Caso afirmativo, o valor **v** está presente na pilha P e, <u>no final</u> da função, o valor *true* deve ser retornado.

     ii. Devolver os elementos armazenados em **AUX** para **P**. Dentro de outro laço de repetição:
        - Desempilhar um elemento da pilha **AUX** e empilhar na pilha **P**.

     iii. Retorna true ou false
   - Exemplo:

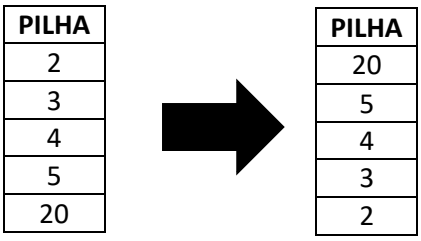
```
bool buscaValor(Pilha *p, int valor)
{
      //passo1 - criar e inicializar a pilha AUX
      //AUX terá o mesmo tamanho da pilha P.

      //passo 2 – desempilhar P e empilhar AUX
      bool encontrou=false;
      while(! vaziaP(p))
      {
            Int v = desempilha(p);
            Empilha(&aux, v);
            If(v == valor)
            {
                  encontrou=true;
                  break;
            }
      }

      //passo 3 - desempilhar AUX e empilhar P
      while(! vaziaP(aux))
      {
            Int v = desempilha(aux);
            Empilha(&p, v);
      }
      return encontrou;
}
```

2. Crie uma função que calcule o maior, o menor e a média aritmética dos valores armazenados em uma pilha.
   - Protótipo da função:
     `void estatisticaP(Pilha *p, int *menor, int *maior, float* media)`
   - A função deve calcular e atualizar os valores das variáveis passadas por referência.
   - Mostrar os dados calculados no programa principal.
   - Se a pilha estiver vazia: menor=0, maior=0 e media=0.

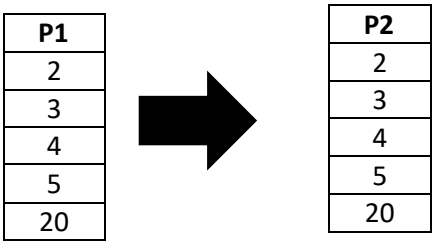
3. Desenvolva uma função para testar se duas pilhas P1 e P2 são iguais.

   Observação: Duas pilhas são iguais se elas possuem os mesmos elementos e na mesma ordem.

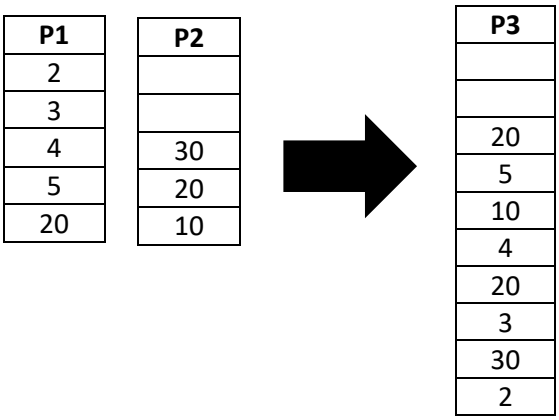
4. Desenvolva uma função para <u>inverter</u> a ordem dos elementos armazenados em uma pilha.
   - Use quantas pilhas auxiliares forem necessárias;
   - Não é permitido o uso de vetores auxiliares;
   - Não é permitido acessar o vetor que armazena os dados da pilha diretamente "pilha->dados[i]", use as funções empilha() e desempilha();
   - Mostar a pilha antes e depois da chamada da função.



5. Crie uma função para <u>copiar</u> os elementos de uma pilha P1 para P2, <u>mantendo a ordem</u> relativa dos elementos.
   - Use quantas pilhas auxiliares forem necessárias;
   - Não é permitido o uso de vetores auxiliares;
   - Não é permitido acessar o vetor que armazena os dados da pilha diretamente "pilha->dados[i]", use as funções empilha() e desempilha();
   - Mostar as pilhas antes e depois da chamada da função.



6. Crie um programa para intercalar duas pilhas P1 e P2 conforme o exemplo abaixo. Note que as pilhas P1 e P2 possuem o mesmo tamanho, mas o número de elementos armazenados em cada uma é diferente.



7. <u>Empregando pilha</u>, crie uma função para verificar se uma expressão matemática composta apenas por parênteses está ou não *balanceada*.
   - Exemplo de expressões balanceadas: "`(A+B)`", "`((A+B)+(C+D))`"
   - Exemplo de expressões não balanceadas: "`((A+B)`", "`)A+B(`", "(A+B)) – (C + D"

8. Um intervalo de valores de uma pilha P é definido por dois números inteiros **I** (início) e **F** (fim), que são os elementos da pilha que estão entre as posições **I** e **F**, sentido topo para base. Considerando duas pilhas estáticas $\mathbf{P_1}$ e $\mathbf{P_2}$, implemente uma função em C++ para verificar se um intervalo de valores de $\mathbf{P_1}$ é igual ao inverso de um intervalo de valores de $\mathbf{P_2}$.

Exemplo:

```
P1: topo [-1, 20, 10, 5] base
Intervalo de P1: I1=1 e F1=2

P2: topo [10, 20, -1, 5] base
Intervalo de P2: I2=0 e F2=1

Intervalos P1[1-2] e P2[0-1] inversos? Sim
```

O protótipo da função deve ser:

```
bool verificaIntervalosInversos(Pilha *p1, Pilha *p2, int i1, int f1, int i2, int f2);
```

O programa deve exibir na tela:

- Imprimir as pilhas $\mathbf{P_1}$ e $\mathbf{P_2}$ antes da chamada da função `verificaIntervalosInversos`.
- Imprimir os intervalos de ambas as pilhas e "sim" se o retorno da função `verificaIntervalosInversos` for verdadeiro, ou "não" caso contrário.
- Imprimir as pilhas $\mathbf{P_1}$ e $\mathbf{P_2}$ depois da chamada da função `verificaIntervalosInversos`.

Observações:

- Utilizar a estrutura de dados Pilha estática na implementação.
- É permitido o uso de pilha auxiliares.
- As pilhas $\mathbf{P_1}$ e $\mathbf{P_2}$ devem permanecer inalteradas, ou seja, os elementos de $\mathbf{P_1}$ e $\mathbf{P_2}$ devem ser os mesmos e na mesma ordem, antes e depois da chamada da função `verificaIntervalosInversos`.
- A função `verificaIntervalosInversos` deve retornar "*false*" se um intervalo for inválido, conforme os seguintes casos:
  - (**I**<0)
  - (**F**<0)
  - (**F**<**I**)
  - (**F**>=**T**) ,sendo **T** o total de elementos da pilha.

9. A conversão de um número decimal N (base 10) para binário (base 2) é realizada por sucessivas divisões de N por 2, em que o número binário é obtido pelos restos das divisões realizadas. Crie um programa para converter número decimais em binários empregando uma pilha.